REPORTE DE PRÁCTICA NO. 6

Análisis sintáctico de expresiones aritméticas

ALUMNOS:

Arrazola Ibarra Juan Pablo
Vazquez Pontaza Fernando
Olivares Roque Diego
Caballero Rossano Gabriel

Dr. Eduardo Cornejo-Velázquez

Introducción

Una Gramática Formal definida sobre un alfabeto Σ es una tupla de la forma:

$$G = \{\Sigma_T, \Sigma_N, S, P\}$$

donde:
Σ_T es un alfabeto de símbolos terminales.
Σ_N es un alfabeto de símbolos no terminales.
S es el símbolo inicial de la gramática.
P es un conjunto de producciones gramaticales.
Además, se cumple:

$$S \in \Sigma_N$$

$$\Sigma_T \cap \Sigma_N = \emptyset$$

$$\Sigma = \Sigma_T \cup \Sigma_N$$

La gramática formal G permite generar un lenguaje:

$$L = \{x \in \Sigma_T^* \mid S \Rightarrow^* x\}$$

Por lo tanto, las palabras del lenguaje estarán formadas por cadenas de símbolos terminales generadas a partir del símbolo inicial de la gramática utilizando las producciones que la definen.
Una expresión regular es una notación normalizada para representar lenguajes regulares, es decir, lenguajes generados por gramáticas regulares (Tipo 3). Las expresiones regulares permiten describir con exactitud y sencillez cualquier lenguaje regular.
El analizador sintáctico es la parte del compilador que se encarga de analizar el código de entrada en base a una gramática dada y, en caso de que el programa de entrada sea válido, construye el árbol sintáctico que lo reconoce.
En teoría, la salida del analizador sintáctico es alguna representación del árbol sintáctico que reconoce la secuencia de tokens suministrada por el analizador léxico; sin embargo, en la práctica, también identifica errores cuando se producen y se comunica con la tabla de símbolos para almacenar información de los procesos realizados.
Un analizador sintáctico es un programa que analiza códigos para determinar su estructura sintáctica.
Ejemplo de análisis sintáctico aritmético: La estructura sintáctica de la cadena $2*3+4$ es el siguiente árbol:

```
    +
   / \
  *   4
 / \
2   3
```

El análisis sintáctico forma parte del preprocesamiento en la mayoría de las aplicaciones reales.
Consideramos expresiones aritméticas:

- construidas con números, operaciones ($+$ y $*$) y paréntesis.
- $+$ y $*$ asocian por la derecha.
- $*$ tiene más prioridad que $+$.

Marco Teórico

El estudio de los lenguajes de programación es muy importante en el área de la computación, ya que permite entender cómo se estructuran y cómo se pueden analizar los programas. Para poder interpretar un código, se utilizan diferentes etapas dentro de lo que se conoce como compilación, entre las más importantes están el análisis léxico y el análisis sintáctico.

El análisis léxico es la primera fase y se encarga de leer el código fuente para dividirlo en partes más pequeñas llamadas tokens. Estos tokens pueden ser números, operadores, identificadores, entre otros. Para realizar este proceso de forma más sencilla, se utilizan herramientas como Lex, que permite definir patrones mediante expresiones regulares.

Después sigue el análisis sintáctico, que utiliza los tokens generados para verificar si la estructura del programa es correcta. En esta etapa se utilizan gramáticas formales para definir cómo deben formarse las expresiones. Una herramienta común para esto es Yacc, la cual permite construir analizadores sintácticos de manera más automatizada.

Las gramáticas formales ayudan a establecer reglas para generar cadenas válidas dentro de un lenguaje. Estas reglas indican cómo se pueden combinar los símbolos para formar expresiones correctas. Además, las expresiones regulares permiten describir patrones de forma sencilla, lo cual es muy útil en el análisis léxico.

En el caso de las expresiones aritméticas, es importante considerar aspectos como la precedencia y la asociatividad de los operadores. Por ejemplo, la multiplicación tiene mayor prioridad que la suma, por lo que se evalúa primero. Esto es fundamental para que los resultados sean correctos.

En general, el uso de herramientas como Lex y Yacc facilita mucho la creación de analizadores, ya que automatizan gran parte del proceso y ayudan a reducir errores al momento de validar estructuras dentro de un programa.

Objetivo General

Desarrollar un analizador léxico y un analizador sintáctico utilizando Lex y Yacc para reconocer y validar operaciones aritméticas, asegurando la detección de errores y la correcta validación de las gramáticas definidas.

Objetivos Específicos

- Emplear las herramientas conceptuales (autómatas y gramáticas libres de contexto) para definir cómo lograr el objetivo de la práctica.
- Diseñar un programa en Lex que identifique números enteros y números decimales, así como símbolos que representen las operaciones de suma, resta, multiplicación y división en el lenguaje Python.
- Implementar un programa en Yacc que verifique la correcta construcción de las operaciones aritméticas mediante una gramática formal.

Desarrollo

Para llevar a cabo esta práctica, se siguieron una serie de pasos con el objetivo de desarrollar un analizador léxico y sintáctico utilizando las herramientas Lex y Yacc.
En primer lugar, se definieron los autómatas necesarios para reconocer los diferentes tokens requeridos en la práctica. Estos tokens incluyen números, operadores y otros elementos básicos que forman parte de las expresiones aritméticas.
Posteriormente, se utilizó la notación BNF (Backus-Naur Form) para describir la gramática de las operaciones aritméticas como suma, resta, multiplicación y división. Esto permitió establecer reglas claras sobre cómo deben formarse las expresiones.
Después, se desarrolló el programa en Lex, el cual se encargó de reconocer los siguientes elementos:

- Números enteros.
- Números decimales.
- Operadores aritméticos como suma (+), resta (-), multiplicación (*) y división (/).
- Símbolos utilizados en expresiones condicionales.
- Identificadores (nombres de variables).
- Paréntesis.

Una vez definido el analizador léxico, se procedió a implementar el programa en Yacc. En esta etapa, se verificó que las expresiones aritméticas cumplieran con las reglas definidas previamente en la gramática. Además, se agregaron acciones para indicar si una expresión era válida o si contenía errores.
Posteriormente, se crearon dos archivos de prueba. El primero contenía expresiones correctas para comprobar el funcionamiento adecuado del programa, mientras que el segundo incluía expresiones con errores sintácticos para verificar que el analizador detectara fallas correctamente.
Finalmente, se ejecutaron los programas desarrollados en Lex y Yacc utilizando ambos archivos. Con esto, se pudieron observar los resultados obtenidos y comprobar el correcto funcionamiento del analizador, tanto en la validación de expresiones correctas como en la detección de errores.
Durante el desarrollo de la práctica, también se documentaron las pruebas realizadas, incluyendo capturas de pantalla y una explicación de los resultados obtenidos, lo cual permitió tener un mejor entendimiento del funcionamiento del sistema.

Proceso de compilación con Lex y Yacc

Para la ejecución del analizador léxico y sintáctico se siguieron los siguientes pasos:

1. Se creó el archivo del analizador léxico `calculator.l`, el cual contiene las reglas para reconocer los tokens.
2. Se ejecutó el comando:

   ```
   lex calculator.l
   ```

   Esto generó el archivo `lex.yy.c`, que corresponde al analizador léxico en lenguaje C.
3. Posteriormente, se creó el archivo `calculator.y`, el cual contiene la gramática y las reglas del analizador sintáctico.
4. Se ejecutó el comando:

   ```
   yacc -d calculator.y
   ```

Esto generó los archivos:

- `y.tab.c`
- `y.tab.h`

los cuales corresponden al analizador sintáctico.

5. Después, se compiló todo con el compilador de C:

```
cc lex.yy.c y.tab.c -o calculator
```

generando el ejecutable `calculator.exe`.

6. Finalmente, se ejecutó el programa para validar las expresiones aritméticas.

Resultados

Durante la ejecución del programa se obtuvieron los siguientes resultados:

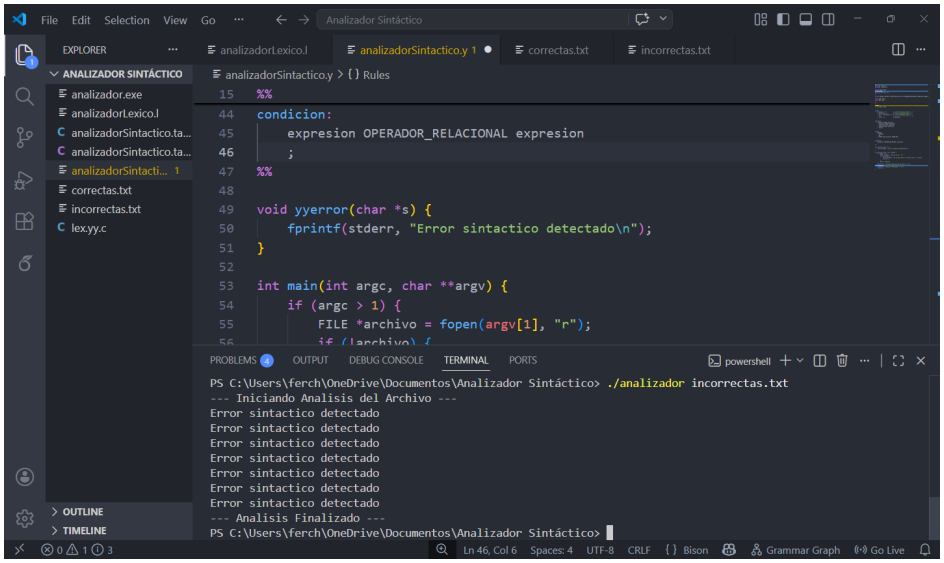

Figure 1: FunciónIncorrecta. Se muestra un ejemplo donde la expresión no cumple con la gramática, por lo que el analizador detecta un error.

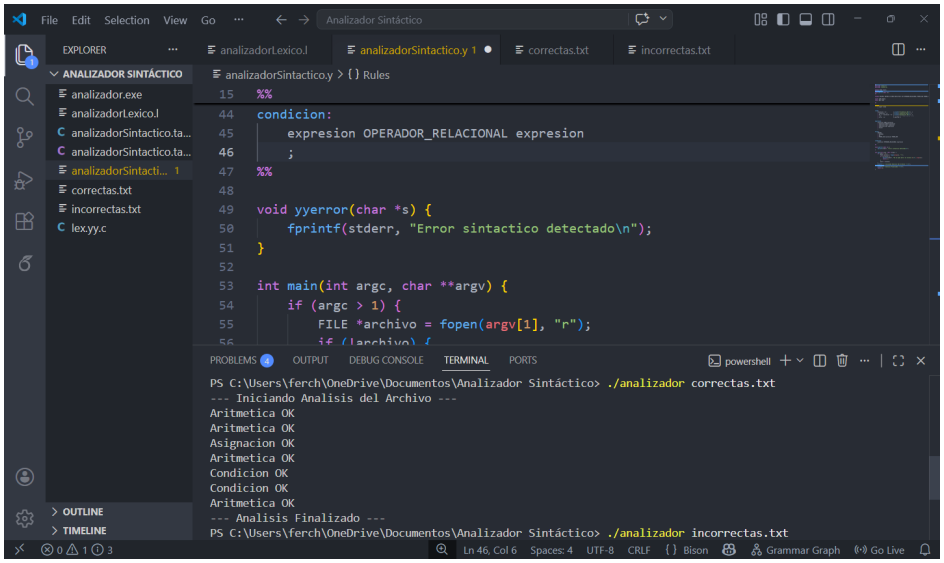

Figure 2: FunciónCorrecta. La expresión cumple con las reglas definidas, por lo que es aceptada correctamente.

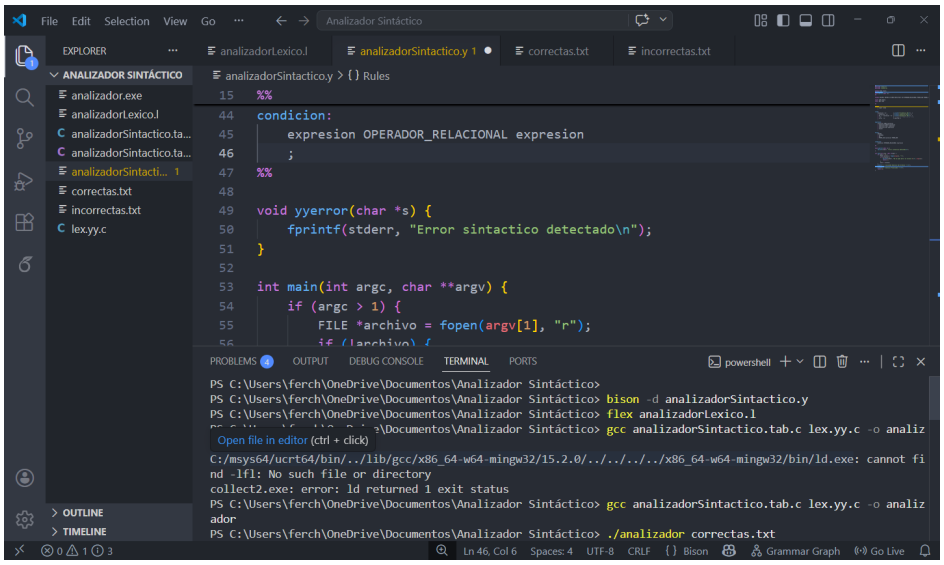

Figure 3: Compilado. Se observa la ejecución del programa integrando Lex y Yacc.

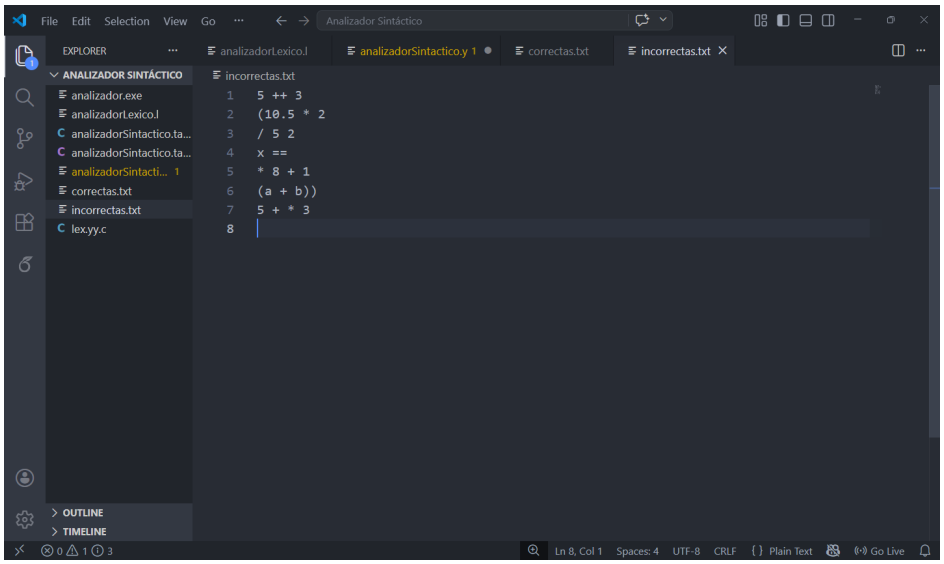

Figure 4: Incorrectas. Archivo con expresiones que contienen errores sintácticos.

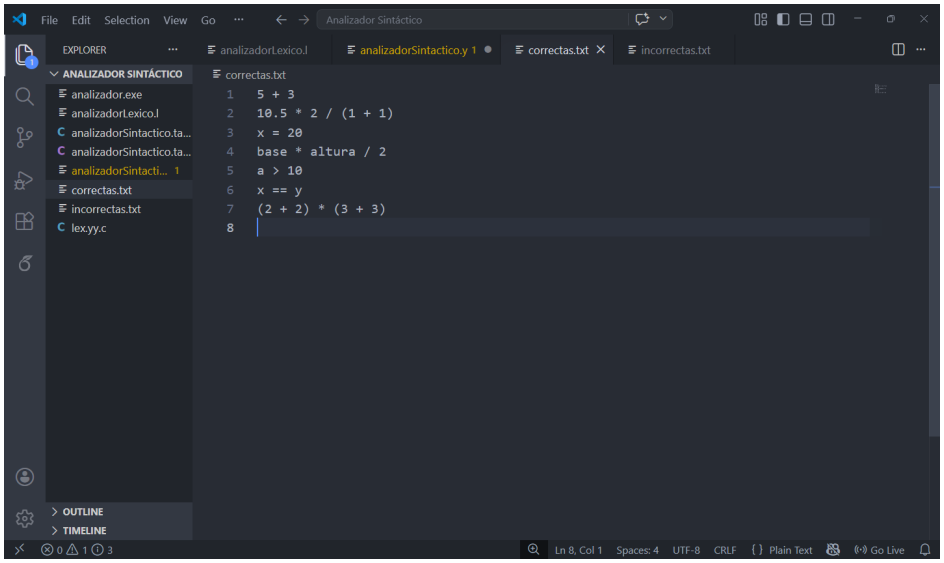

Figure 5: Correctas. Archivo con expresiones válidas que el analizador reconoce correctamente.

```
%%
condicion:
    expresion OPERADOR_RELACIONAL expresion
    ;
%%

void yyerror(char *s) {
    fprintf(stderr, "Error sintactico detectado\n");
}

int main(int argc, char **argv) {
    if (argc > 1) {
        FILE *archivo = fopen(argv[1], "r");
        if (!archivo) {
            fprintf(stderr, "No se pudo abrir el archivo %s\n", argv[1]);
            return 1;
        }
        yyin = archivo;
    }
    printf("--- Iniciando Analisis del Archivo ---\n");
    yyparse();
    printf("--- Analisis Finalizado ---\n");
    return 0;
}
```

Figure 6: Yacc-3. Parte final del código del analizador sintáctico.

```
%%
    expresion '\n'          { printf("Aritmetica OK\n"); }
    | condicion '\n'        { printf("Condicion OK\n"); }
    | ID '=' expresion '\n' { printf("Asignacion OK\n"); }
    | '\n'                  { /* Ignorar lineas vacias */ }
    | error '\n'            { yyerrok; }
    ;

13 references
expresion:
    expresion SUMA expresion
    | expresion RESTA expresion
    | expresion MULT expresion
    | expresion DIV expresion
    | factor
    ;

1 reference
factor:
    ENTERO
    | DECIMAL
    | ID
    | PAREN_IZQ expresion PAREN_DER
    ;
```

Figure 7: Yacc-2. Definición intermedia de la gramática.

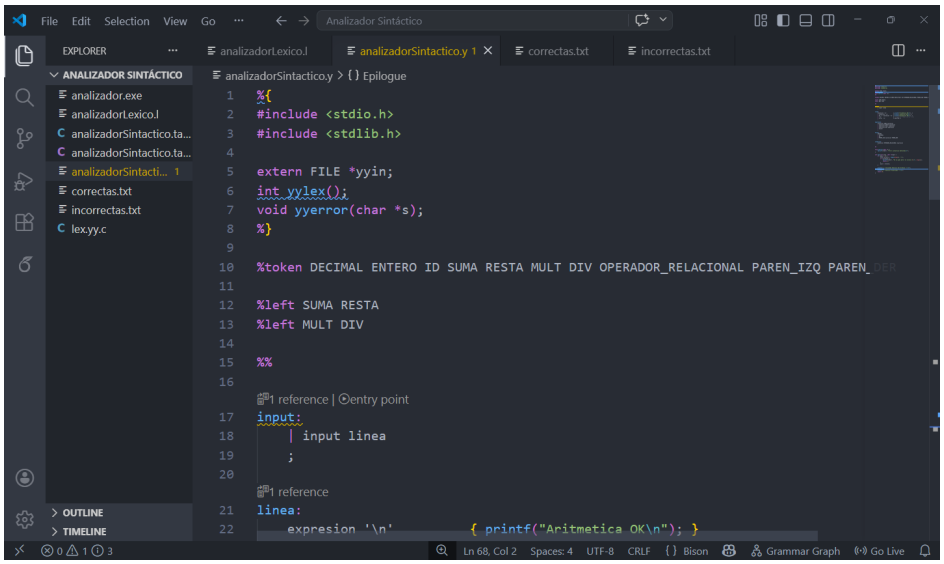

Figure 8: Yacc-1. Estructura inicial del analizador sintáctico.

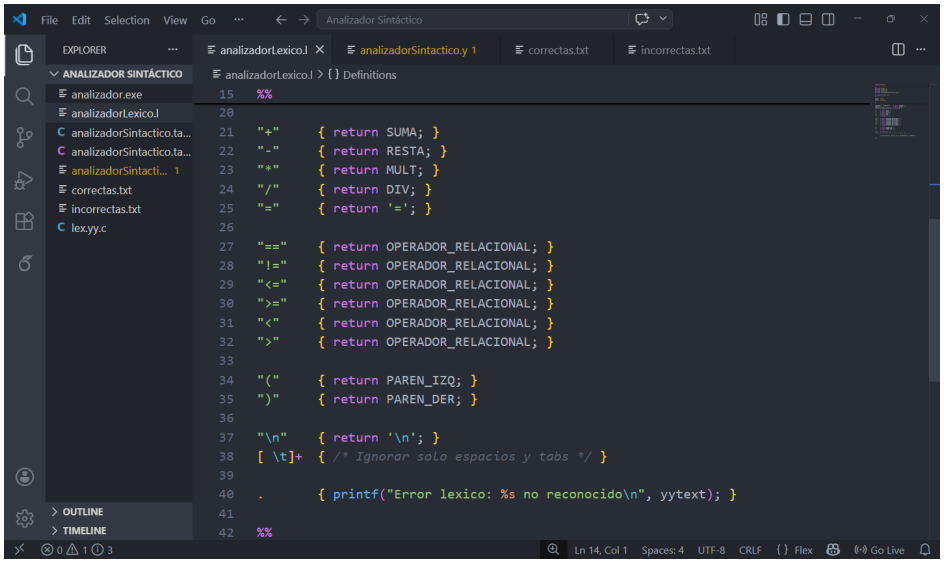

Figure 9: Lex-2. Reglas adicionales para el reconocimiento de tokens.

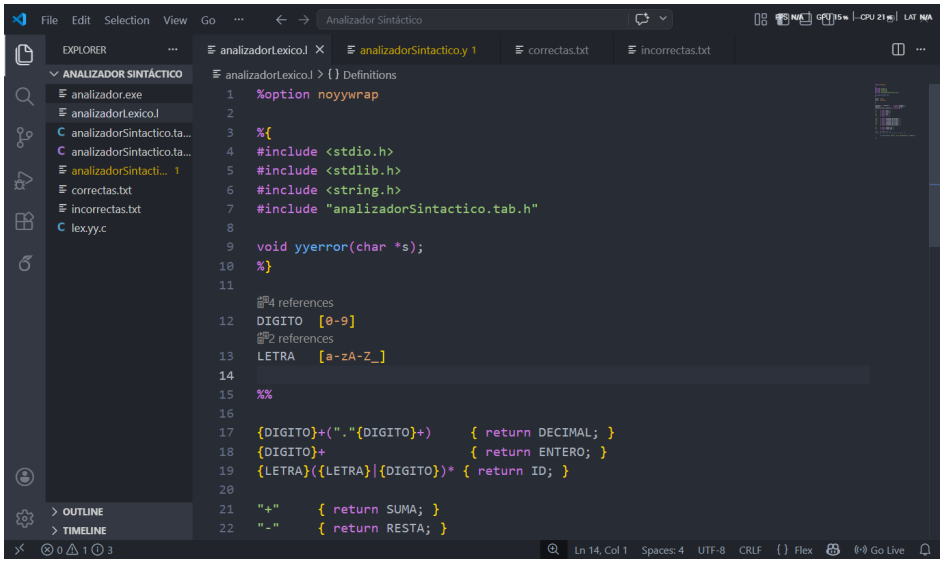

Figure 10: Lex-1. Definición de tokens básicos como números y operadores.

Cuestionario

¿Qué tipo de errores fueron detectados en los ejemplos incorrectos y cómo influyeron las gramáticas definidas para lograr su identificación?
En los ejemplos incorrectos se detectaron principalmente errores sintácticos, como el uso incorrecto de operadores, falta de operandos, paréntesis mal colocados o expresiones incompletas. Las gramáticas definidas fueron fundamentales para identificar estos errores, ya que establecen reglas claras sobre cómo deben formarse las expresiones. Cuando una cadena no cumple con estas reglas, el analizador sintáctico la rechaza automáticamente.

¿Cómo contribuye el uso de derivaciones por la izquierda o por la derecha en la construcción de un árbol sintáctico para validar las operaciones y las prioridades de las operaciones?
Las derivaciones por la izquierda o por la derecha ayudan a construir el árbol sintáctico paso a paso, lo cual permite entender cómo se interpreta una expresión. Esto es importante porque asegura que se respeten las prioridades de los operadores, por ejemplo, que la multiplicación se evalúe antes que la suma. Gracias a estas derivaciones, se puede verificar que la estructura de la expresión sea correcta y que el resultado sea el esperado.

¿Qué ajustes serían necesarios en la gramática para permitir el reconocimiento de estructuras condicionales más complejas, como $(a+b)^2$, raíz cuadrada, así como operaciones trigonométricas?
Para permitir el reconocimiento de estructuras más complejas, sería necesario ampliar la gramática agregando nuevas reglas. Por ejemplo, se tendría que incluir el operador de potencia, funciones como raíz cuadrada y funciones trigonométricas como seno, coseno o tangente. También sería importante definir su precedencia y la forma en que se escriben, para que el analizador pueda interpretarlas correctamente dentro de las expresiones.

Conclusiones

A lo largo de esta práctica se logró comprender mejor el funcionamiento de los analizadores léxicos y sintácticos, así como su importancia en el procesamiento de lenguajes de programación. El uso de herramientas como Lex y Yacc permitió observar de manera práctica cómo se pueden reconocer y validar expresiones aritméticas.
También se entendió la utilidad de las gramáticas formales y la notación BNF para definir reglas claras en la construcción de expresiones. Esto facilitó la implementación del analizador sintáctico y ayudó a identificar cuándo una expresión es válida o presenta errores.
Durante el desarrollo, se pudo comprobar que el analizador funciona correctamente al procesar tanto expresiones correctas como incorrectas, lo cual es importante para detectar errores en programas reales. Además, el uso de archivos de prueba permitió validar el comportamiento del sistema de manera más completa.
En general, la práctica ayudó a reforzar conceptos teóricos vistos en clase y a aplicarlos en un entorno práctico, lo que permitió tener una mejor comprensión del tema.

Bibliografía

References

[1] Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.

[2] Gutú, O. (2013). *Primer curso en teoría de autómatas y lenguajes formales*. Pearson Educación. México.

[3] Hopcroft, J. E., Motwani, R., Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación*. Pearson Educación. México.